Team Responsibility

GraphGuard: Capstone System Hardening & Maturity Report

Team: Dynamic Agents

1. **System Reproducibility & Stability Overview**

   The objective of this hands-on sprint was to eliminate "works on my machine" vulnerabilities and elevate GraphGuard to a production-ready standard. We achieved this through strict environment isolation and execution automation. We implemented a run.sh script that serves as our single-command deployment mechanism. This script orchestrates dependency installation, executes a mandatory pre-flight smoke test (scripts/smoke_test.py), and gracefully aborts if the environment is unhealthy, ensuring the Streamlit application never launches in a broken state. To guarantee ultimate reproducibility, we also containerized the entire pipeline using Docker.

2. **Environment Control & Configuration Discipline**

   We utilized agentic IDE tools to accelerate our system refactoring, specifically focusing on decoupling our application logic from hardcoded parameters. The agentic assistant analyzed our legacy app/main.py and successfully extracted hardcoded paths, model names, and latency thresholds into a centralized config/settings.json file. This config-driven approach allows external DevOps pipelines to modify system behavior without altering the underlying Python source code. Furthermore, we stabilized our Python environment by addressing Python 3.13 compatibility conflicts within our requirements.txt, ensuring deterministic builds.

3. **Engineering Audit & Stress Test Results**

   We conducted a rigorous stress test by executing a clean clone on a fresh machine environment without any pre-existing virtual environments. The audit revealed critical deployment breakpoints:

- Hidden Dependency Failures: Git inherently ignores empty directories. On a fresh clone, the application immediately crashed with a FileNotFoundError because the local logs/ directory did not exist to receive the telemetry data.
- Module Pathing Errors: Restructuring our application into a modular format (agent/agent_runner.py) initially caused ModuleNotFoundError crashes during

Streamlit execution because Python could not resolve the internal directory paths.

- Resolution: We introduced robust error handling. The smoke_test.py script now automatically verifies the existence of required directories and dynamically provisions them. Module pathing was resolved by enforcing proper **init**.py initializations and restructuring the Streamlit execution context to correctly append the root directory to the system path.

4. **Logging and Artifact Structure**

   We hardened our observability pipeline into a fully automated system. The Streamlit dashboard now features an integrated "Pipeline Monitoring" tab. Every interaction with the AI agent automatically calculates execution latency and writes the telemetry (User ID, Latency, Status) to logs/pipeline_logs.csv. We also established an artifacts/ directory designed to capture stateful outputs, ensuring that all agent decisions remain fully auditable.

Individual Engineering Reflection

Name: Prathyu Adari

Project: GraphGuard - Dynamic Identity Verification Agent

Role: Product Intelligence, Strategy & Insight Lead

Engineering Contribution & Agentic AI Automation

For our Week 7 system hardening sprint, my primary contribution was orchestrating the deployment environment and integrating our disparate system components into a single, production-ready application. I utilized agentic IDE tools to assist in generating boilerplate deployment scripts, including our Dockerfile and run.sh files. I also used the AI to help refactor our Streamlit application (app/main.py), successfully merging our AI Agent chat interface, interactive Snowflake visualizations, and real-time pipeline monitoring into a unified, 3-tab dashboard. While the agentic tools generated the structural code, I spent the majority of my time manually debugging the environment configurations, file paths, and dependency conflicts required to actually make the system run locally.

Discovered Failures and Resolutions

During the engineering audit and execution of our deployment scripts, I encountered and systematically resolved a series of cascading environment failures:

1. **Dependency & Compiler Conflicts:** My initial attempt to run pip install -r requirements.txt failed because I accidentally created a directory named requirements instead of a text file. After correcting this, I hit a massive C++ compiler failure. I discovered this was because we were using Python 3.13, and the strictly pinned version of Pandas (2.2.0) lacked a pre-built wheel for this new Python version. I resolved this by unpinning the dependencies in requirements.txt, allowing pip to dynamically resolve the latest compatible versions.

2. **Configuration Parsing Crash:** When I first ran our smoke_test.py script, the application crashed with a JSONDecodeError: Expecting value: line 1 column 1. I investigated and found that while the config/settings.json file existed, it was completely empty. I fixed this by properly hydrating the JSON structure with our latency targets and file paths.

3. **Module Routing and Import Failures:** When launching Streamlit, I hit multiple ModuleNotFoundError crashes. First, Python couldn't find the agent module because the folder was missing an __init__.py file. Second, agent_runner.py was sitting in the root directory instead of the agent/ folder. Finally, once moved, the app crashed again because of a circular import (import GraphGuardAgent

written inside the file where the class itself was defined). I debugged and resolved each of these pathing and import issues sequentially.

4. **OS-Level Extension Strictness:** I also struggled with Windows Notepad forcing a .txt extension onto our Dockerfile, which breaks containerization. I learned how to bypass OS-level defaults by saving the file with explicit quotation marks ("Dockerfile") to drop the extension.

System Design Evolution and Key Learnings

This hands-on week completely changed my perspective on what it means to build a "production AI system." Previously, my design focus was almost entirely on the LLM—tuning the prompt and ensuring the RAG pipeline retrieved the correct NIST guidelines. Today's debugging session taught me that a highly intelligent model is completely useless if the surrounding infrastructure is fragile. Fixing folder structures, resolving Python version conflicts, handling empty JSON files, and managing strict file extensions showed me that engineering maturity is really about anticipating and controlling the environment your code runs in.